

SOLID 检查清单

AI 设计缺陷注入实验

法律文书标注平台 · 架构设计审查

2025

目 录

- 一、S — 单一职责原则
- 二、O — 开闭原则
- 三、L — 里氏替换原则
- 四、I — 接口隔离原则
- 五、D — 依赖倒转原则
- 六、SOLID 违规汇总表
- 七、统计
- 八、原始类图（按模块分页）
- 九、修改方案

一、S — 单一职责原则 (Single Responsibility Principle)

检查问题：有没有类承担了过多职责？

结论：违反，共发现 3 处。

类名	涉及职责数	具体职责列表	违反说明	修正方案
TaskAssigner	5	① 任务生命周期管理（创建、阶段推进） ② 人员分配（assignAnnotator、assignArbitrator） ③ 标签配置管理（labelConfigSnapshot 字段） ④ 数据导出（exportData） ⑤ 多维度状态判断（canAdvance、isLabelSystemLocked）	一个类同时负责"任务自身数据"、"人员分配流程"、"配置版本控制"、"导出协调"四大不同领域的逻辑，任何一个领域的变更都可能导致 Task 类被修改，引起不必要的回归风险。	拆分为 5 个专门类/服务： ① Task 保留核心属性 + 生命周期方法 ② 新增 TaskAssignmentService 处理分配逻辑 ③ 新增 LabelConfigService + LabelConfig 实体处理配置历史 ④ 新增 ExportCoordinator 委托导出流程 ⑤ 状态判断逻辑下沉到状态模式各子类

一、S — 单一职责原则 (Single Responsibility Principle)

An not ati on	3	① 数据实体（存储命题/关系快照） ② 业务行为（submit、overwrite） ③ 序列化/反序列化（getPropositions、getRelations）	数据实体类混入业务逻辑和数据转换逻辑，违反了分层设计原则。序列化方式变更（如从 JSON 改为 Protocol Buffers）会迫使数据实体类修改。	拆分为两层： ① <code>Annotation</code> 仅保留数据属性（POJO/Entity） ② 新增 <code>AnnotationService</code> 处理 submit/overwrite 业务逻辑 ③ 新增 <code>AnnotationDataConverter</code> 负责 JSON ↔ 对象转换
Us er	3	① 用户数据载体 ② 认证逻辑（login、logout） ③ 权限判断（hasPermission） ④ 任务查询（getTasks）	领域模型类（User）混入横切关注点（认证、权限）和业务查询逻辑，导致领域层与基础设施层耦合。	拆分： ① <code>User</code> 仅保留数据属性 ② 新增 <code>AuthService</code> 处理 login/logout ③ 新增 <code>PermissionService</code> 处理 hasPermission ④ <code>getTasks</code> 移至 <code>TaskQueryService</code>

二、O — 开闭原则 (Open/Closed Principle)

检查问题：新增需求类型是否需要修改现有代码？

结论：违反，共发现 4 处。

场景	是否违反	违反说明	修正方案
新增导出格式（如增加 XML 导出）	是	原 <code>Task.exportData(format)</code> 方法需要直接修改 <code>Task</code> 类来支持新格式，违反"对扩展开放"。	使用模板方法模式： ① 定义抽象类 <code>AbstractExporter</code> ② 各格式（Excel/JSON/PNG/SVG）继承并实现 <code>transform()</code> 和 <code>generateFile()</code> ③ <code>Task</code> 仅持有 <code>Exporter</code> 接口引用，新增格式零侵入
新增任务角色（如增加"审核员"角色）	是	原 <code>Task</code> 类中 <code>assignAnnotator()</code> 和 <code>assignArbitrator()</code> 是两个独立硬编码方法，新增角色需再增加一个方法。	使用统一入口： ① 改为 <code>assignMember(userId, documentId, roleType: Enum)</code> 单方法 ② 角色类型通过 <code>TaskRole</code> 枚举扩展 ③ 或进一步使用策略模式，不同角色对应不同 <code>AssignmentStrategy</code>
标签配置历史追溯	是	原 <code>Task</code> 类用 <code>labelConfigSnapshot: JSON</code> 字段存储配置，是一个"死字段"。新增"查看配置变更历史"功能时，必须将 JSON 字段重构为独立实体，修改 <code>Task</code> 类结构。	提取独立实体 <code>LabelConfig</code> （已修正）： ① <code>LabelConfig</code> 与 <code>Task</code> 为 1:N 关联 ② 通过 <code>LabelConfigService</code> 管理版本历史 ③ <code>Task</code> 类不再直接管理配置数据

二、O — 开闭原则 (Open/Closed Principle)

新增文件解析
格式（如增加
HTML 导入）

轻
度

原类图中虽然定义了 `DocumentParser` 接口和
`PdfParser / WordParser / TextParser` 三个实现，但
接口设计尚未完全落实，存在退化风险——如果开发者在
`TaskDocument` 中直接根据文件类型做 if-else 判断选择解
析器，则开闭原则被破坏。

确保

`DocumentParser` 接
口正确使用：

① `TaskDocument` 持

有 `DocumentParser`

接口引用而非具体类

② 通过

`DocumentParserFa`

`ctory` 在运行时根据

文件类型动态选择策略

③ 新增 HTML 格式只

需新增 `HtmlParser`

`implements`

`DocumentParser`

三、L — 里氏替换原则 (Liskov Substitution Principle)

检查问题：子类是否可以替换父类使用？

结论：轻度违反，共发现 1 处。

类/继承关系	是否违反	违反说明	修正方案
Label 类同时用于一级标签和二级标签	轻度违反	原设计用一个 Label 类表达两种概念：一级标签（SF/GF/SM/GM）和二级标签（GM-L/GM-I 等）。通过 level 字段和 parentL1Id 字段区分。这导致： ① 一级标签的 parentL1Id 字段永远为空/无意义，存在语义污染 ② 调用方必须在使用前判断 isLevel2()，无法无差别替换使用 ③ ER 图中明确区分"一级标签表"和"二级标签表（仅GM）"，类图应尊重这一语义边界	方案 A（推荐）：保留统一 Label 类，但通过工厂方法保证创建时语义正确： ① Label.createL1(...) 创建时不传 parentL1Id ② Label.createL2(...) 创建时必须填 parentL1Id ③ 文档层面明确说明 parentL1Id 对 L1 标签无意义 方案 B（更严格）：拆分为 L1Label extends Label 和 L2Label extends Label，通过继承表达层级差异

注：原类图中没有大量继承层次结构，所以里氏替换的违规点相对较少。上述问题属于"语义层面"的替换障碍，而非典型的继承 override 破坏前置/后置条件。

四、I — 接口隔离原则 (Interface Segregation Principle)

检查问题：有没有接口太"胖"，包含了不需要的方法？

结论：违反，共发现 2 处。

接口/类	是否违反	违反说明	修正方案
------	------	------	------

隐含的 `Task` 是原 `Task` 类的方法集包含了生命周期、分配、配置、导出四大不相关领域的方法。如果将其行为抽象为接口，下游依赖方会被迫依赖大量不需要的的方法：

- ① "任务列表查询"只需要 `getStatus()`、`getTitle()` 等读取方法
- ② "人员分配"只需要 `assignMember()` 相关方法
- ③ "导出功能"只需要 `exportData()` 相关方法

将 `Task` 的行为拆分为多个细粒度接口：

- ① `TaskReadable`
— 读取任务基本信息
 - ② `TaskLifecycle`
— 阶段推进相关
 - ③ `Assignable`
— 人员分配相关
 - ④ `Configurable`
— 标签配置相关
 - ⑤ `Exportable`
— 数据导出相关
- 不同依赖方只依赖所需接口

`DocumentParser` 潜在风险
原接口只有 `parse()` 和 `extractReasonSection()` 两个方法，当前尚不算"胖接口"。但如果未来增加更多方法（如 `validateFormat()`、`extractMetadata()`、`getPageCount()`），则对于纯文本解析器来说，`getPageCount()` 等方法将是无意义的。

- 预防性拆分：
- ① `DocumentParser`
— 只保留 `parse()`
 - ② `ReasonExtractable`
— 保留 `extractReasonSection()`
 - ③ `PageCountable`
— 新增 `getPageCount()`
- 各解析器按需实现多个接口

五、D — 依赖倒转原则 (Dependency Inversion Principle)

检查问题：高层模块是否直接依赖了低层模块的具体实现？

结论：违反，共发现 3 处。

高层模块	直接依赖的低层模块	是否违反	违反说明	修正方案
Task (领域层)	ExportLog (具体类)	是	Task.exportData(): ExportLog 直接返回 ExportLog 具体类，而非抽象。当导出逻辑变更（如异步导出、分片导出）时，领域层 Task 被迫修改。	引入接口：Task 依赖 ExportService 接口： <pre>interface ExportService { export(taskId, format): ExportResult; }</pre> ExportLog 降为 ExportResult 的一种实现
Annotation (实体层)	JSON 序列化机制	是	Annotation.getPropositions() / getRelations() 内部直接进行 JSON 反序列化，实体层直接依赖了数据格式细节。JSON 格式变更或切换序列化方案时，实体类必须修改。	依赖倒置：实体层只持有 String / byte[] 原始数据，反序列化由 AnnotationDataConverter 服务完成，实体通过 Converter 接口间接获取转换结果
Task (领域层)	labelConfigSnapshot: JSON	是	领域对象 Task 直接依赖了 JSON 数据格式的具体存储方式，配置存储方式变更（如改为独立表存储）时，Task 类结构被迫修改。	提取 LabelConfig 独立实体，Task 通过 LabelConfigRepository 接口获取配置数据，Task 本身不再持有配置存储细节

六、SOLID 违规汇总表

SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
S — 单一职责	Task类是否承担了过多职责？	违反	Task类同时负责任务生命周期、人员分配、标签配置管理、数据导出、状态判断5大类职责，任何一类变更都可能修改Task类	拆分为 Task（核心数据）+ TaskAssignmentService + LabelConfigService + ExportCoordinator
S — 单一职责	Annotation类是否承担了过多职责？	违反	数据实体类混入业务逻辑（submit/overwrite）和数据转换逻辑（JSON序列化/反序列化）	拆分为 Annotation（纯数据实体）+ AnnotationService（业务逻辑）+ AnnotationDataConverter（数据转换）
S — 单一职责	User类是否承担了过多职责？	违反	领域模型类混入认证（login/logout）、权限（hasPermission）、业务查询（getTasks）三类横切/业务逻辑	拆分为 User（纯数据）+ AuthService + PermissionService + TaskQueryService
O — 开闭原则	新增导出格式是否需要修改现有代码？	违反	<code>Task.exportData(format)</code> 需要在Task类中增加对新格式的处理逻辑	使用模板方法模式： <code>AbstractExporter</code> 定义骨架，各格式继承实现
O — 开闭原则	新增任务角色是否需要修改现有代码？	违反	<code>assignAnnotator()</code> 和 <code>assignArbitrator()</code> 是两个硬编码方法，新增角色需增加新方法	统一为 <code>assignMember(uid, docId, roleType: Enum)</code> 或策略模式
O — 开闭原则	新增配置历史追溯功能是否需要修改现有代码？	违反	<code>labelConfigSnapshot</code> 是 JSON死字段，增加历史版本功能需重构Task类结构	提取独立的 <code>LabelConfig</code> 实体类，1:N关联Task

六、SOLID 违规汇总表

O — 开 闭 原 则	新增文件解析格式是否需要修改现有代码?	轻度风险	虽然定义了DocumentParser接口, 但存在退化风险 (可能在TaskDocument中做if-else)	确保通过 DocumentParserFactory 动态选择策略, 不侵入业务代码
L — 里 氏 替 换	Label类用于两种标签层级是否有替换问题?	轻度违反	一级标签的 parentL1Id 字段无意义, 调用方必须先判断 isLevel2() 才能安全使用	方案A: 通过工厂方法 (createL1/createL2) 确保创建语义正确; 方案B: 拆分为 L1Label/L2Label继承Label
I — 接 口 隔 离	是否存在太"胖"的接口?	违反	Task类的方法集横跨生命周期/分配/配置/导出四个领域, 若抽象为接口则依赖方被迫依赖不需要的方法	拆分为 TaskReadable / TaskLifecycle / Assignable / Configurable / Exportable 等细粒度接口
I — 接 口 隔 离	DocumentParser接口未来是否可能变胖?	潜在风险	当前尚合理, 但未来增加 validateFormat/getPageCount 等方法后, 将成为胖接口	预防性拆分为 DocumentParser / ReasonExtractable / PageCountable 等独立接口
D — 依 赖 倒 转	Task领域层是否直接依赖低层具体实现?	违反	Task.exportData(): ExportLog 直接依赖ExportLog具体类, 而非导出服务抽象	引入 ExportService 接口, Task依赖接口而非具体实现
D — 依 赖 倒 转	Annotation实体层是否直接依赖序列化细节?	违反	JSON反序列化逻辑硬编码在实体类中, 实体直接依赖数据格式	将反序列化逻辑移至 AnnotationDataConverter, 实体通过接口间接获取

六、SOLID 违规汇总表

D — 依赖倒转	Task是否直接依赖JSON存储方式？	违反	<code>labelConfigSnapshot:</code> JSON 使领域对象直接依赖了具体存储格式	提取 <code>LabelConfig</code> 独立实体，Task通过 Repository 接口获取配置
----------	---------------------	----	--	---

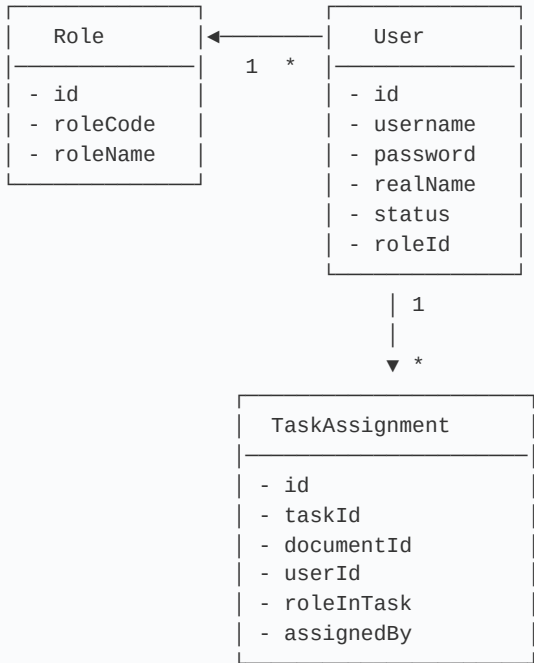
七、统计

统计项	数量
SOLID 违规总数	13 项
严重违反	7 项 (S:3, O:3, D:3)
轻度违反	2 项 (L:1, O:1)
潜在风险	2 项 (O:1, I:1)
已修正项	1 项 (提取 LabelConfig, 同时修正 O 和 D 各 1 项)
待修正项	12 项

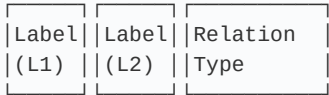
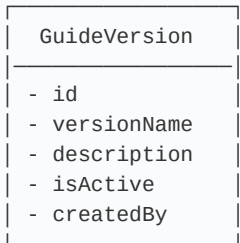
八、原始生成 — 类图（按模块分页）

以下为 AI 生成的原始类图，按功能模块分页面展示，便于逐模块审查 SOLID 合规性。

【用户权限模块】



【配置中心模块】



Label 二级特有: parentL1Id →

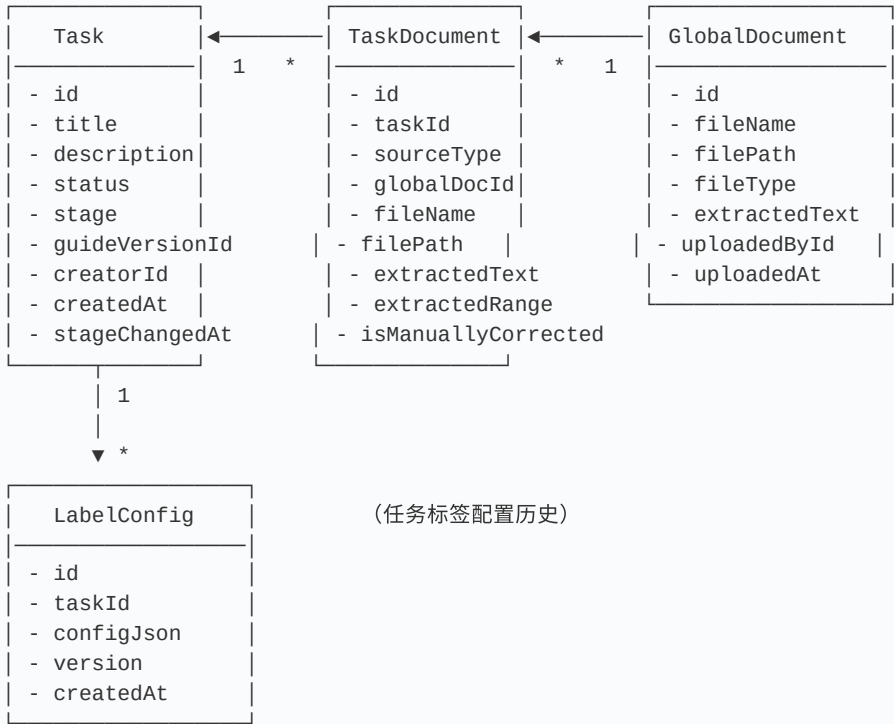
RelationType:

- id
- guideVersionId
- name
- abbr
- isBinary
- sortOrder

Label (一级标签 / 二级标签共用)

- id
- guideVersionId
- name
- abbr
- sortOrder
- parentL1Id (二级标签特有)

【任务管理模块】



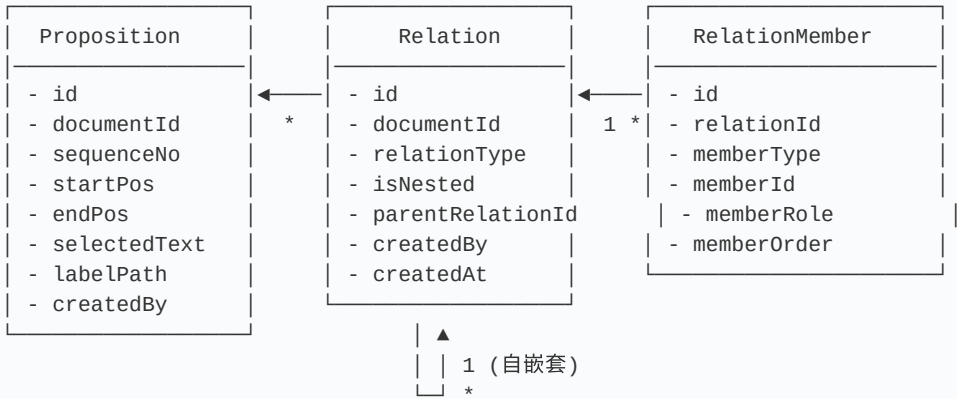
(任务标签配置历史)

Task 与 User 关系:

Task.creatorId → User.id (多对一, 创建者)

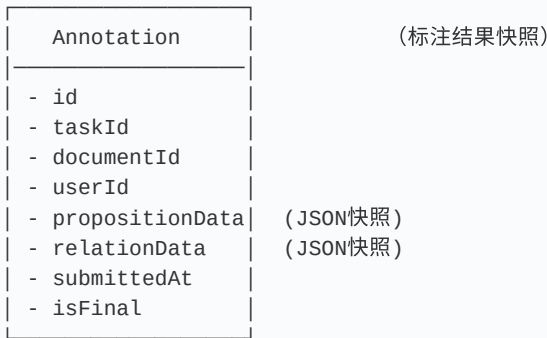
Task.guideVersionId → GuideVersion.id (多对一)

【标注核心模块】

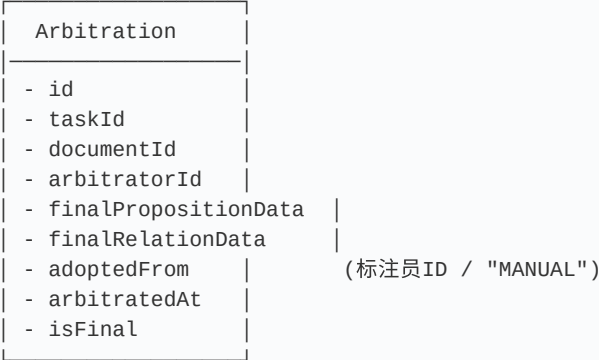


说明：

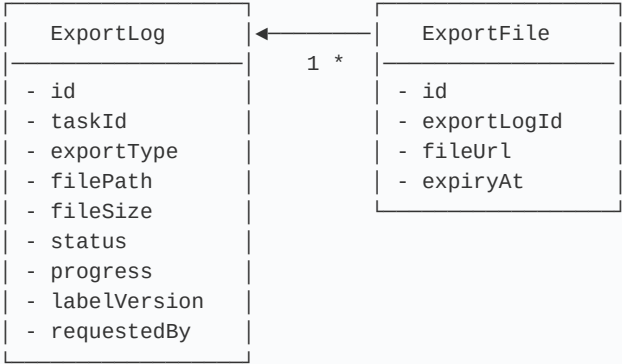
- Proposition 与 Relation 之间为多对多，通过 RelationMember 关联
- RelationMember.memberType 区分 'P'(命题) / 'R'(关系)
- Relation 支持自嵌套 (parentRelationId)，实现关系组合



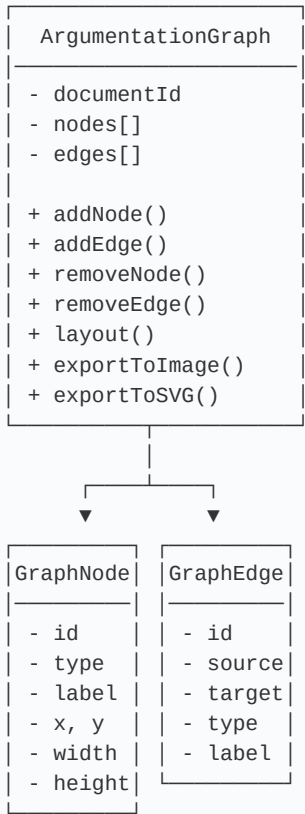
【冲突裁决模块】



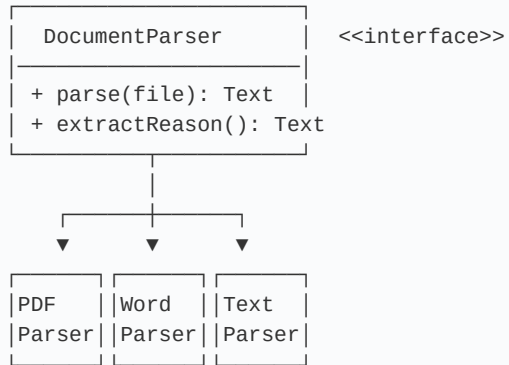
【数据导出模块】



【论证图示模块】



【文本处理模块】



九、修改方案

#	原则	违规点	修正措施
1	S	Task 5大职责聚合	拆出 <code>TaskLifecycleService</code> 、 <code>TaskAssignmentService</code> 、 <code>LabelConfigService</code> 、 <code>ExportCoordinator</code>
2	S	Annotation 混入业务+序列化	拆出 <code>AnnotationService</code> 、 <code>AnnotationDataConverter</code>
3	S	User 混入认证+权限+查询	拆出 <code>AuthService</code> 、 <code>PermissionService</code> 、 <code>TaskQueryService</code>
4	O	新增导出格式改现有代码	模板方法模式： <code>AbstractExporter</code> + 子类
5	O	新增角色需加硬编码方法	<code>assignMember(uid, docId, roleType)</code> 统一入口
6	O	labelConfigSnapshot JSON 死字段	提取 <code>LabelConfig</code> 独立实体
7	O	Parser退化风险	<code>DocumentParserFactory</code> 强制走接口
8	L	Label类语义双重性	工厂方法 <code>createL1()</code> / <code>createL2()</code> 保证创建语义
9	I	Task方法横跨4领域	拆分为 <code>ITaskLifecycle</code> / <code>IAssignable</code> / <code>IExportable</code> 等
10	I	Parser未来变胖风险	拆分为 <code>IDocumentParser</code> / <code>IReasonExtractable</code>
11	D	Task直接依赖ExportLog	Task → <code>IExportService</code> → <code>ExportCoordinator</code> → <code>ExportLog</code>
12	D	Annotation依赖JSON序列化	JSON转换移至 <code>AnnotationDataConverter</code> ，实体只存原始字符串
13	D	Task依赖JSON存储格式	<code>labelConfigSnapshot</code> 移除，改为 <code>LabelConfig</code> 实体